

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

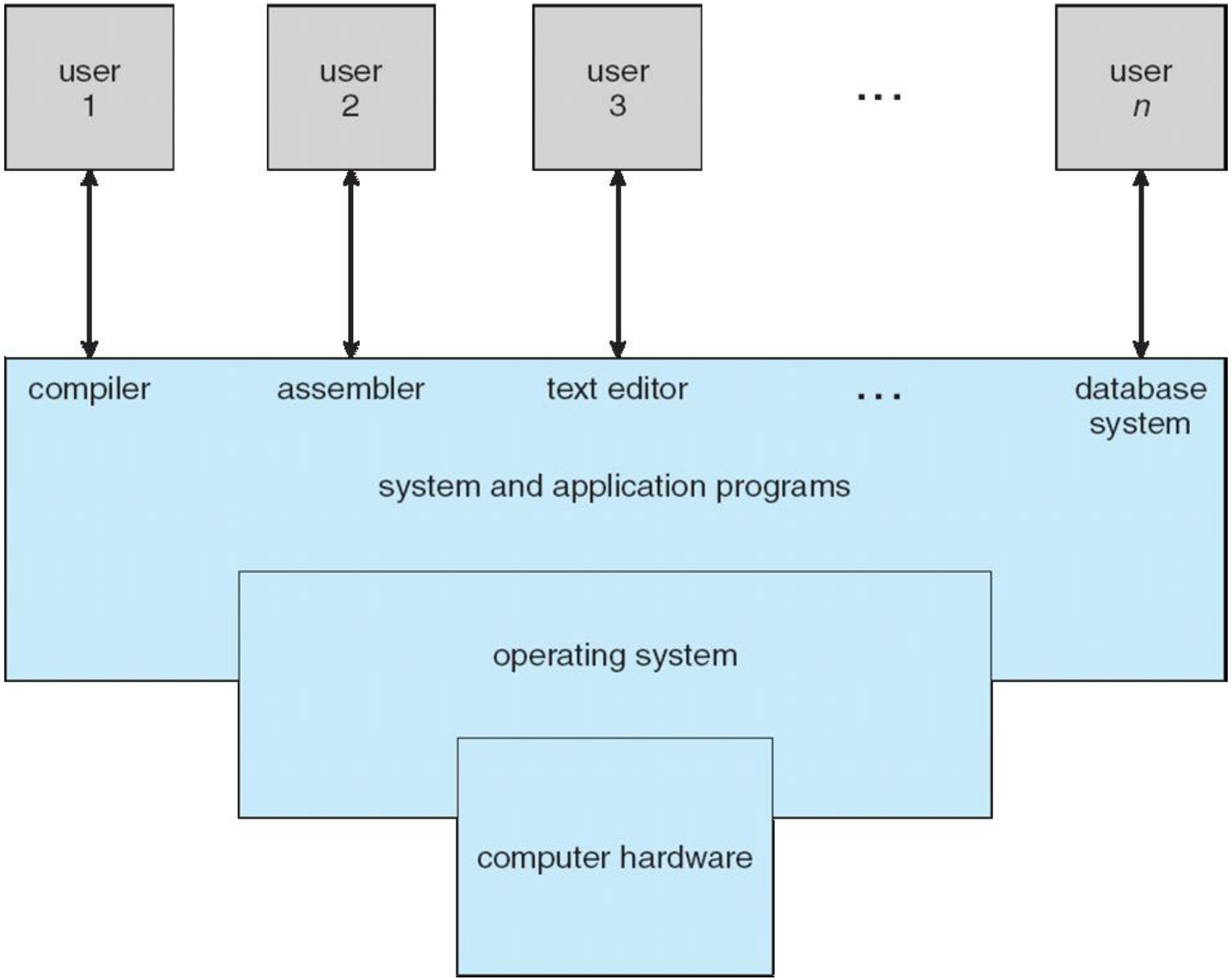
- Lab 0 due this Friday (soft deadline; not graded)
- Homework 1 will be posted this Friday
- Recitations start this week
- VS Code setup tutorial on Piazza (also linked from Canvas)
- Draft Slides linked from Canvas

Agenda

- Main tasks of an operating system
- System Calls
  - What an interrupt is
  - What happens when an interrupt occurs
  - What a system call is
  - How system calls implemented
  - Effect of OS structure on system calls

What is an Operating System?

A program that acts as an intermediary between a user of a computer and the computer hardware



What does an OS do?

- Manages (controls and arbitrates) resources
  - Processors, Memory, Input/output devices, Communication devices, Storage, Software applications
  - Conflicting goals:
    - e.g., performance *vs.* utilization
    - Separation of policy and mechansim
- Provides abstractions to application programs
  - Ease of use
  - Virtualization
- Protects resources

Interrupts

Hardware or software:

- Hardware interrupt by one of the devices
- Software interrupt (**exception** or **trap):**
  - Software error (e.g., division by zero)
  - Other process problems include processes trying to modify each other's or the operating system's memory (e.g., segmentation fault)
  - Request for operating system service (i.e., system call)

Interrupt Descriptor Table

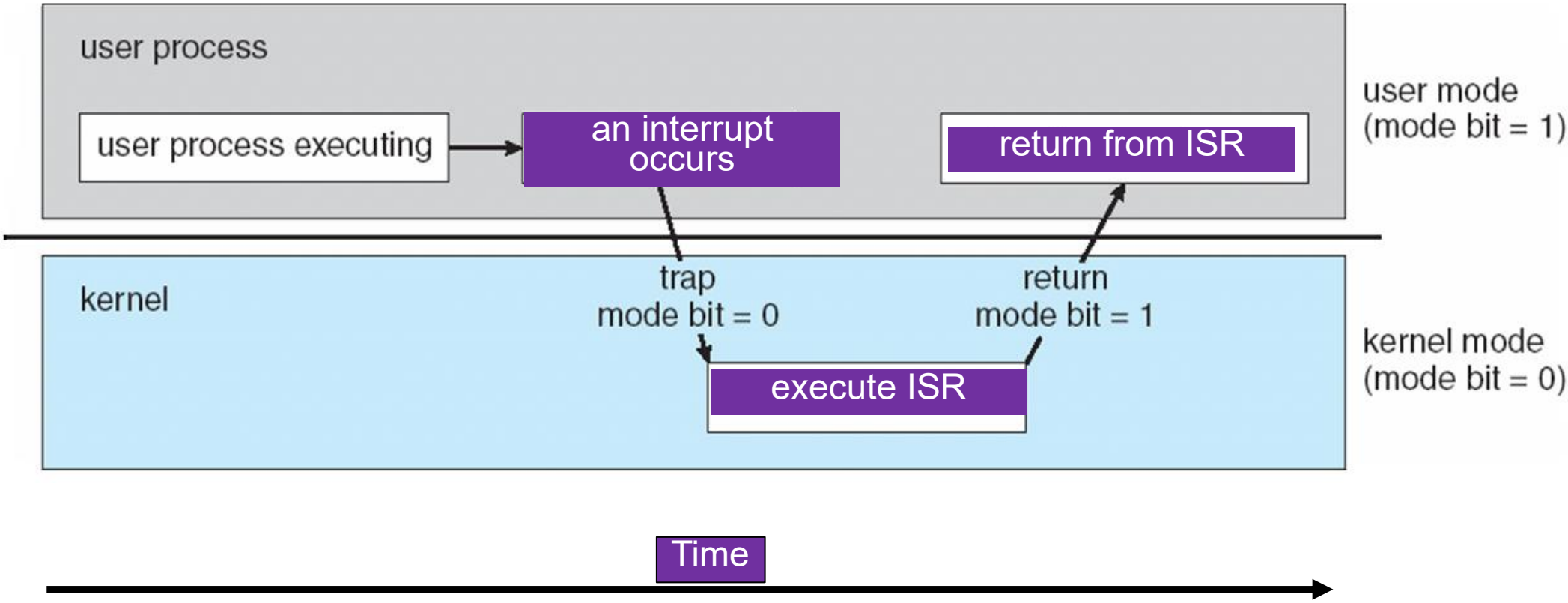
- Interrupt transfers control to the interrupt service routine (ISR)
- ISRs are segments of code that determine what action should be taken for each type of interrupt
  - part of the OS kernel
- An **interrupt vector** contains the address of the ISR for one interrupt
- An **interrupt vector table** is an array of interrupt vectors
  - **also known as interrupt descriptor table (IDT)**

Dual-mode Operation

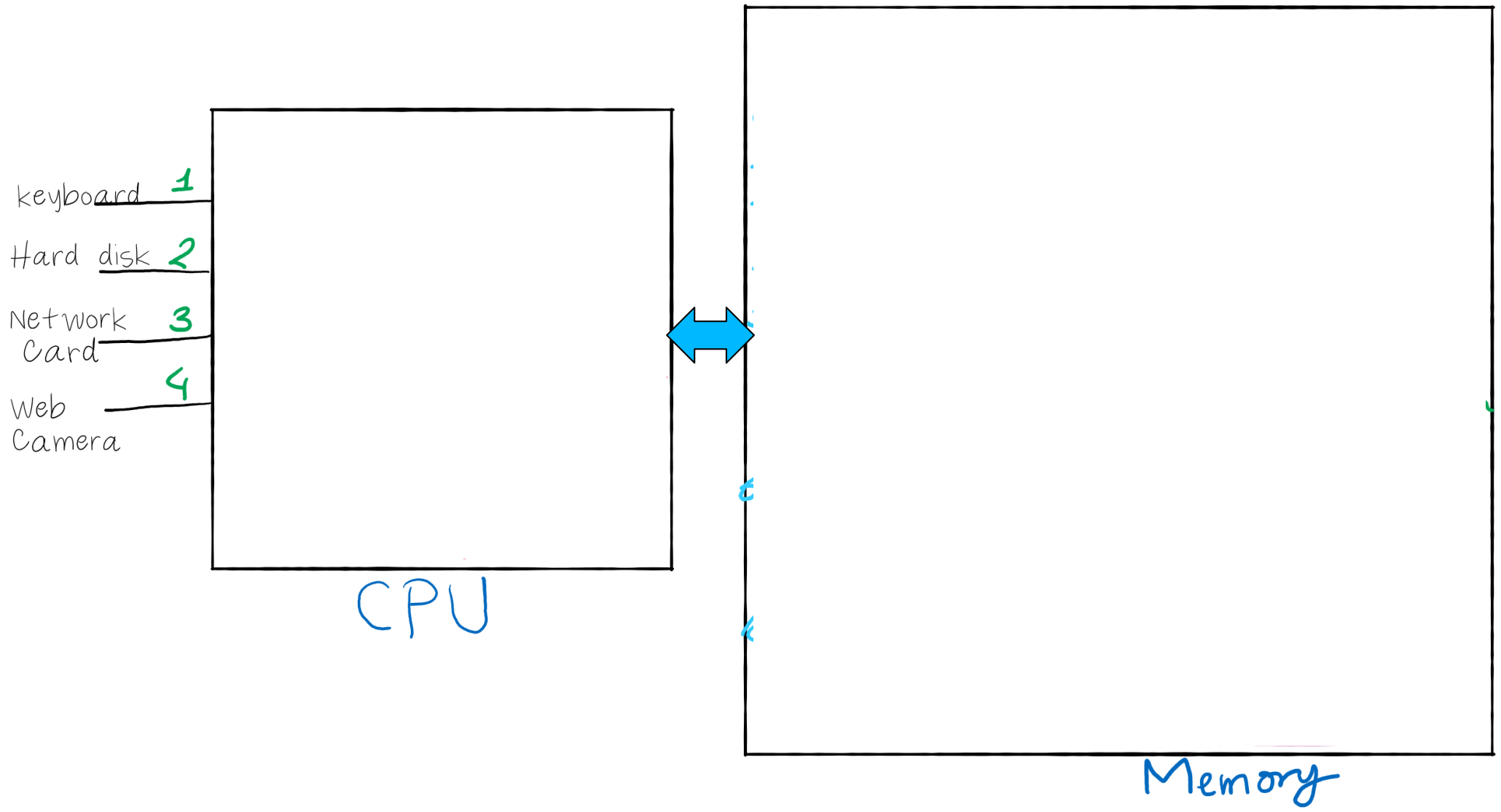
- **Dual-mode** operation allows OS to protect itself and other system components
  - **At least two modes: user mode** and **kernel mode**
  - **Mode bit(s)** provided by hardware (inside CPU registers)
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - Some memory addresses designated as **privileged**, only accessible in kernel mode
      - Therefore, we get segmentation fault on null (i.e., 0) pointer dereference
    - Interrupts change mode to kernel
      - return from interrupt resets mode back to user
- Increasingly CPUs support multi-mode operations
  - **virtual machine manager** (**VMM**) mode for guest **VMs**

What happens when an interrupt occurs?

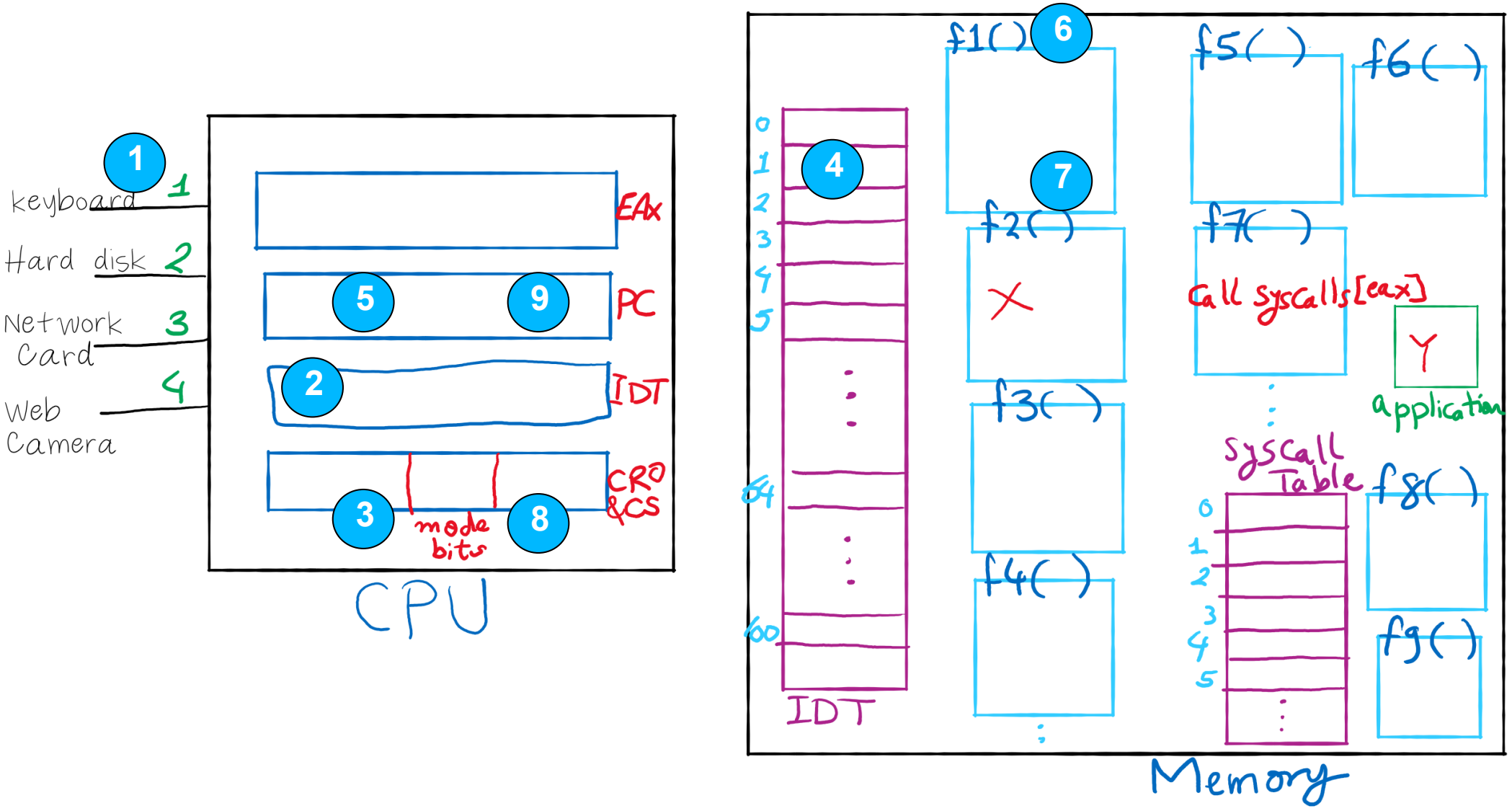
The CPU transitions from User Mode to Kernel Mode



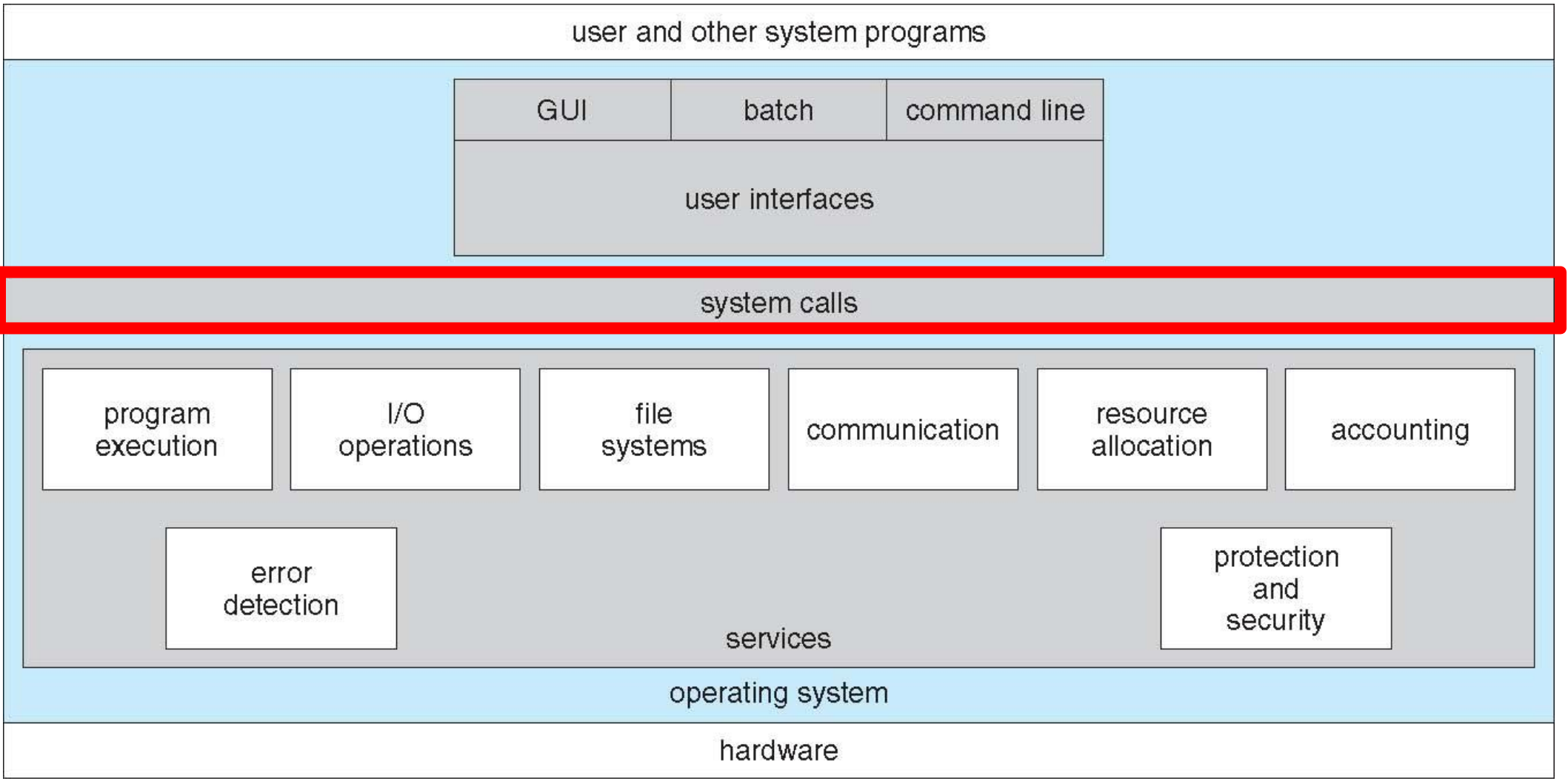
How does an OS (roughly) work?



What happens on a hardware interrupt?



System Calls



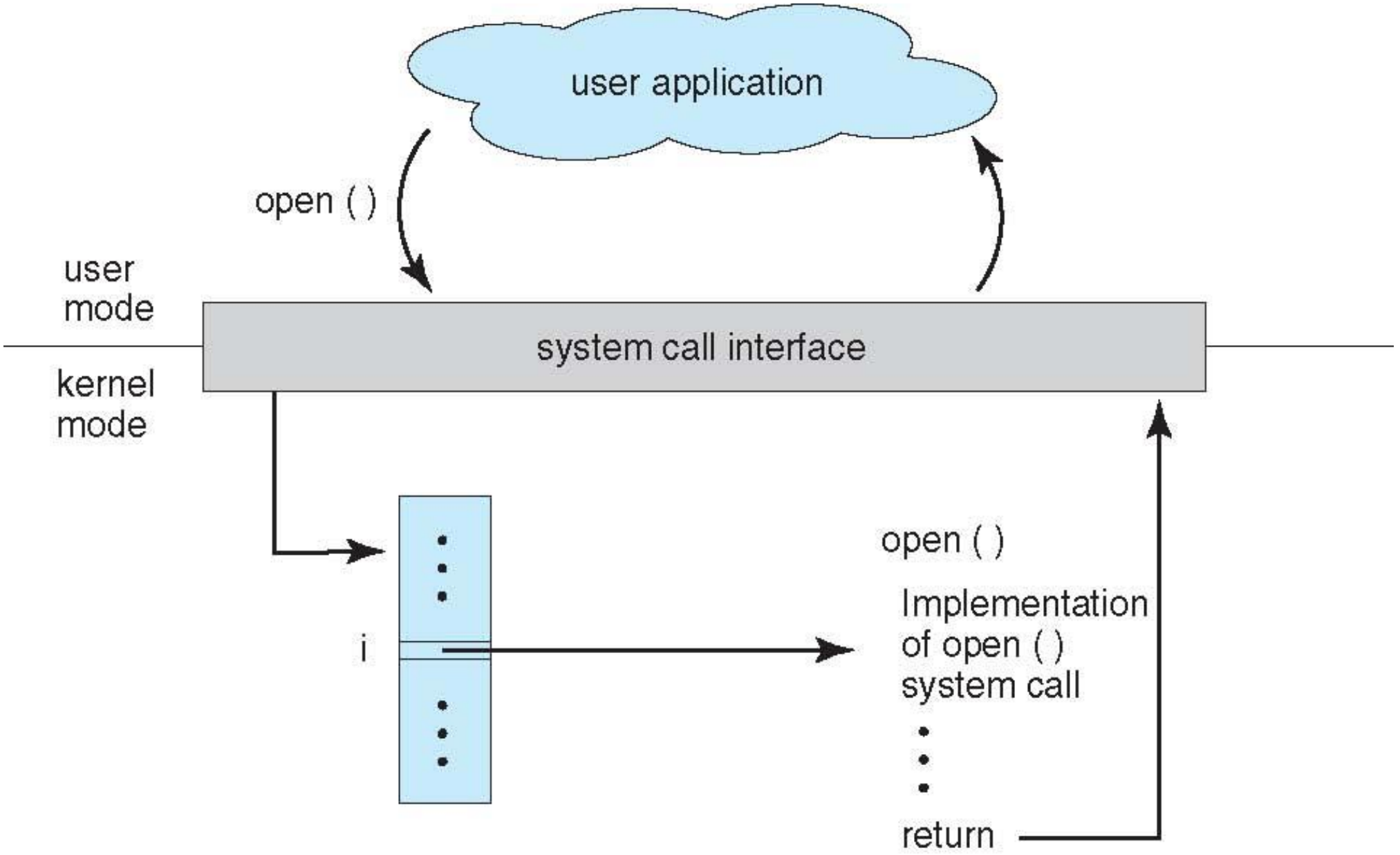
System Calls

- Programming interface to OS services
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
  - Win32 API for Windows
  - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and
  - Java API for the Java virtual machine (JVM)

System Call Implementation

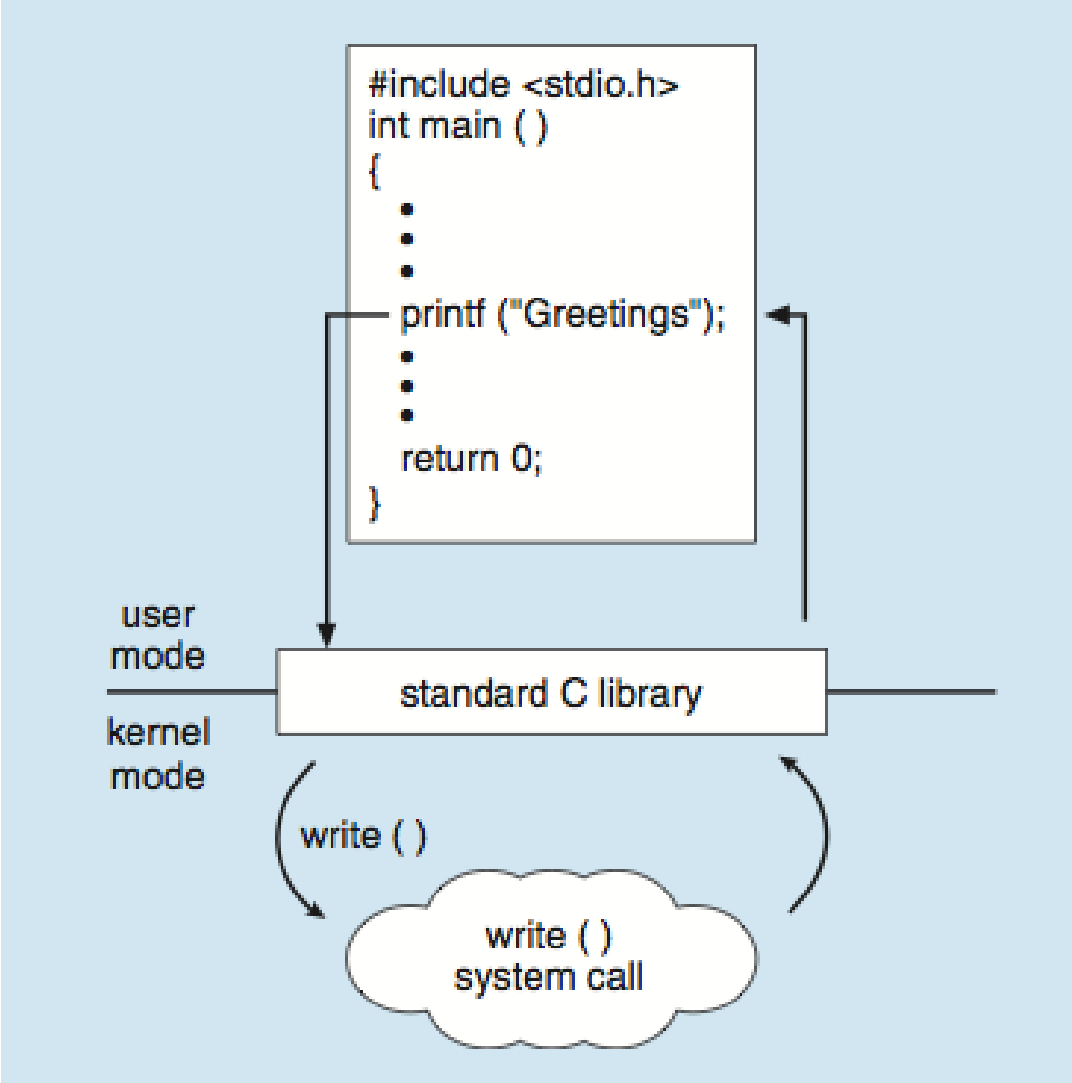
- Typically, there is a number associated with each system call
- Each system call has a corresponding system call implementation function (part of the OS kernel)
- **System-call table** indexed according to these numbers
  - Each entry in the table contains the address of the corresponding system call implementation function
- The system call interface is the ISR corresponding to the syscall software interrupt
  - invokes the intended system call in OS kernel,
  - passes arguments if needed, and
  - returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented

API – System Call – OS Relationship

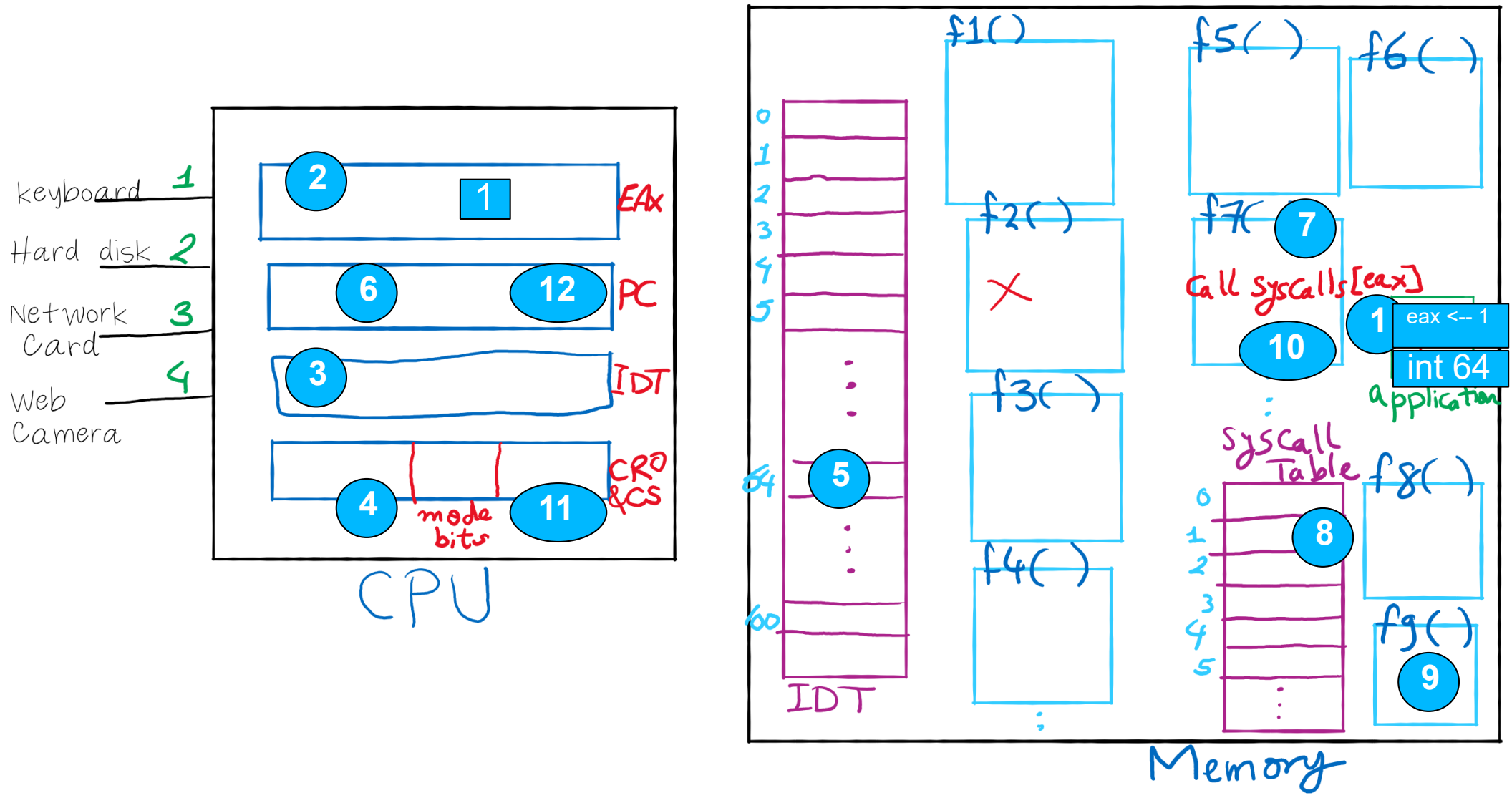


Standard C Library Example

- C program invoking printf() library call, which calls write() system call



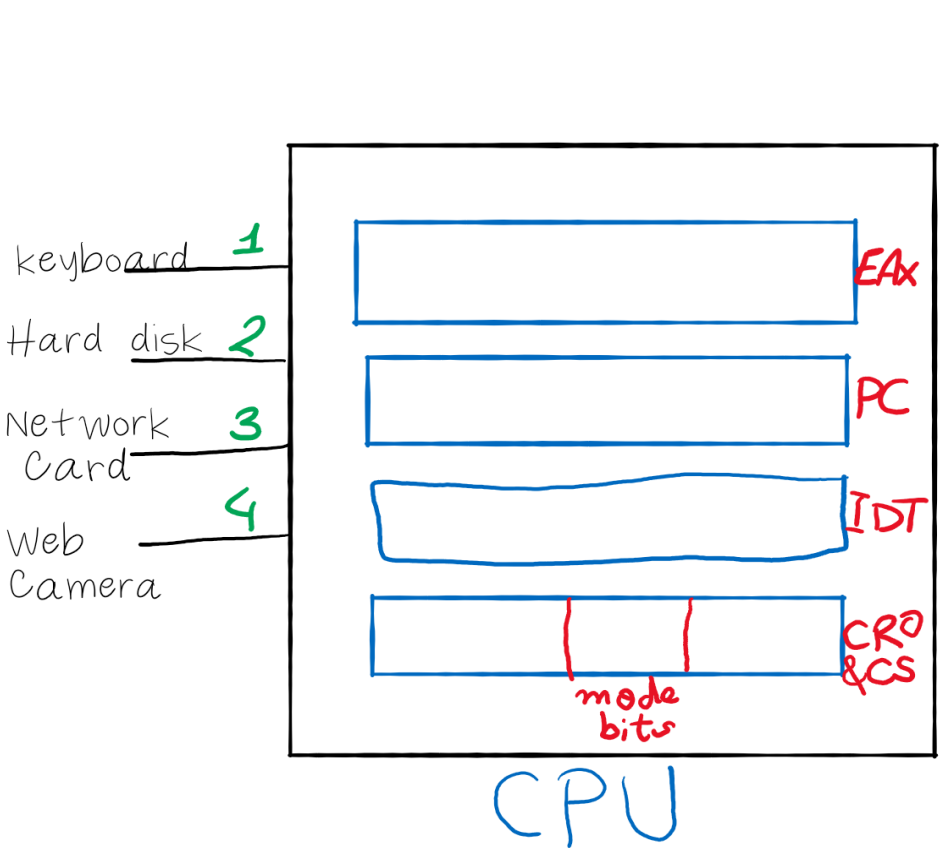
What happens on a syscall?

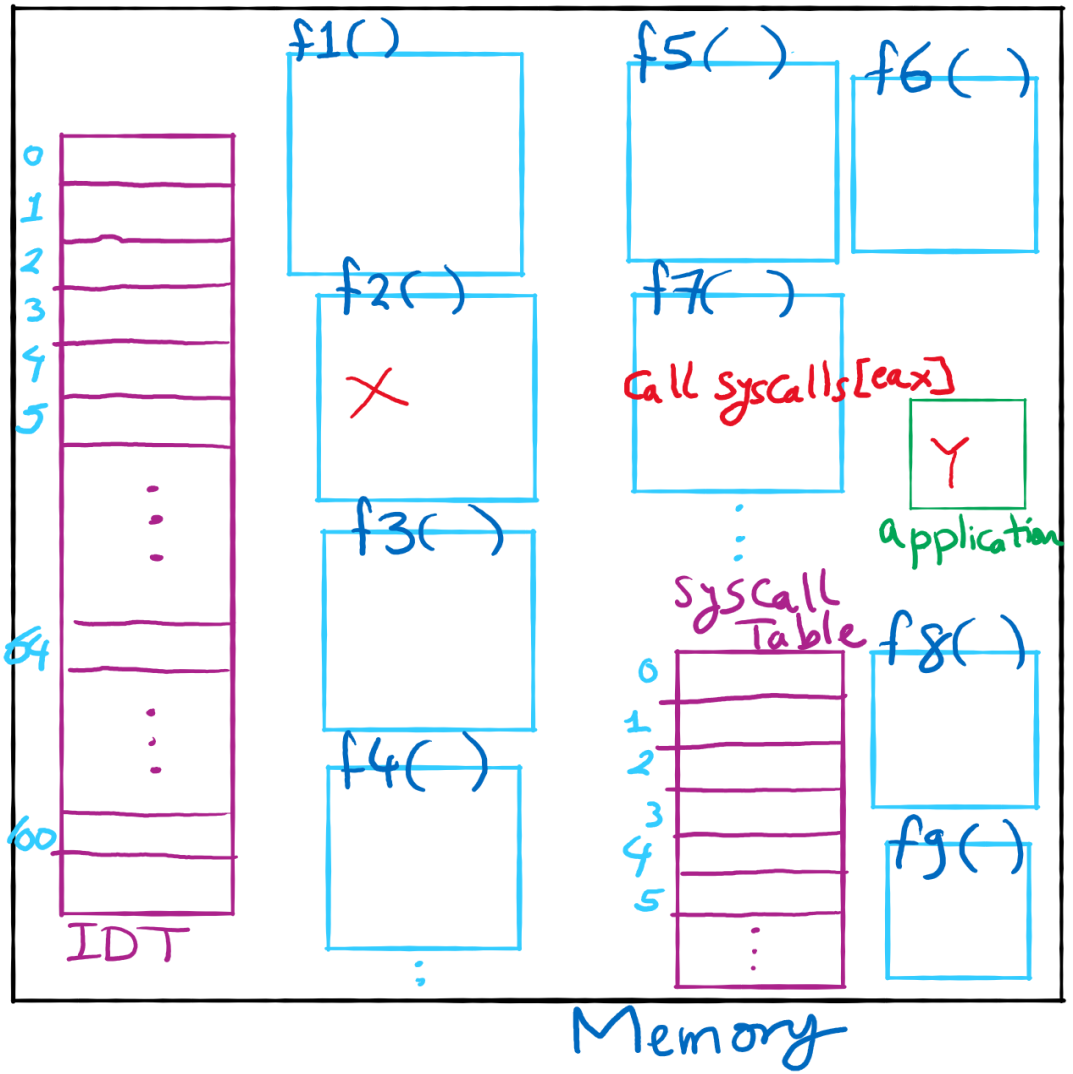


System Call Parameter Passing

- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
    - XV6
- Block and stack methods do not limit the number or length of parameters being passed

How to add a system call to an OS?



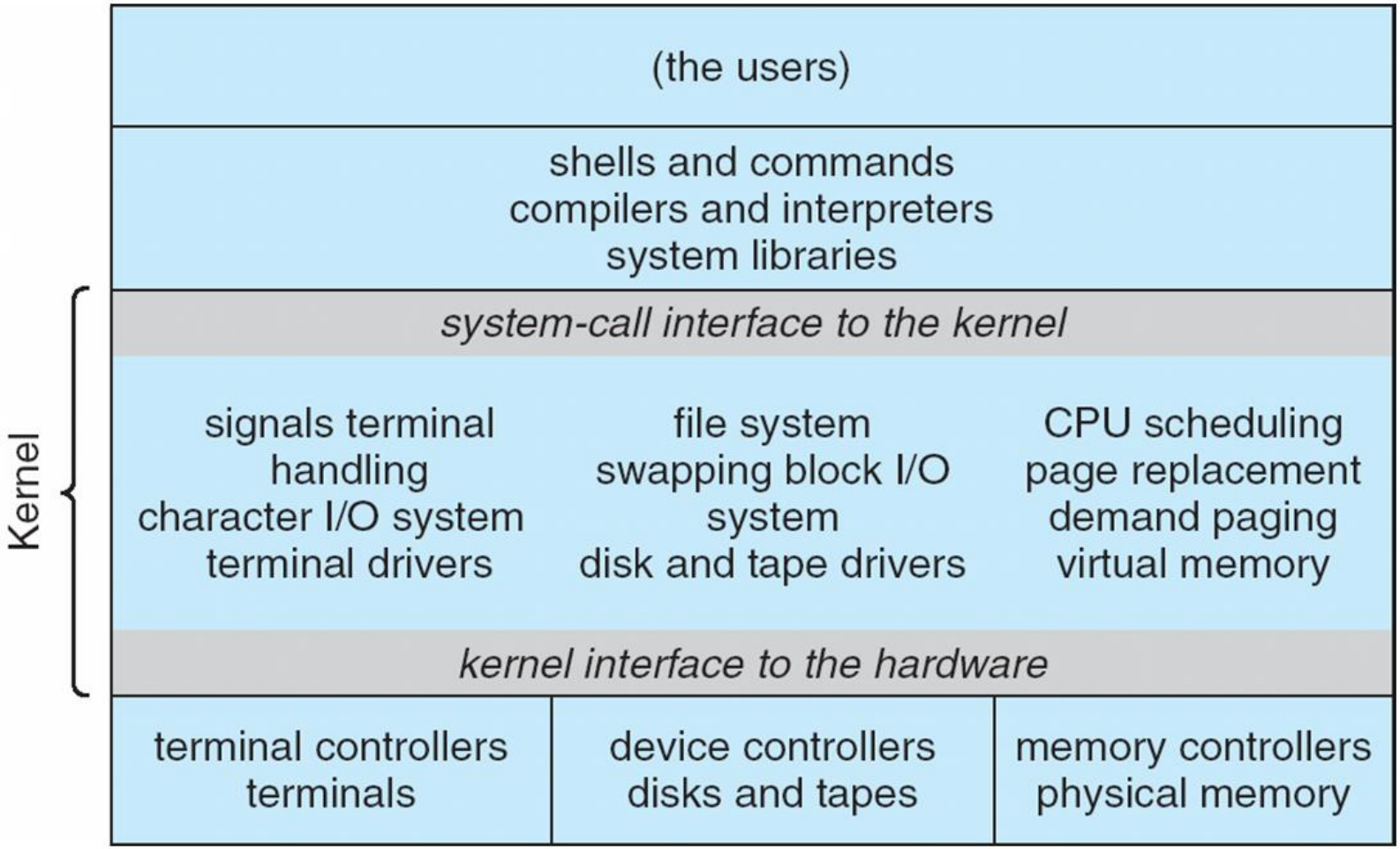


Xv6 Code Walkthrough

- IDT table initialization
- Syscall table
- How a syscall is invoked
- Syscall implementation
- Parameter passing into a syscall
- In Lab 1 you will add a system call to Xv6

Traditional UNIX System Structure

Beyond simple but not fully layered



Microkernel System Structure

